

LAB 02 REPORT

GROUP MEMBERS: BURAIR HAIDAR 09628

SUALEH ABBAS 10303

task 0:

The screenshot shows the STM32CubeIDE interface. The main editor displays the `main.c` file with the following code:

```

1  int main(void)
2  {
3      /* Infinite loop */
4      /* USER CODE BEGIN WHILE */
5      while (1)
6      {
7          /* USER CODE END WHILE */
8          char msg[] = "Hello, World!\r\n";
9          HAL_UART_Transmit(&huart2,
10                           (uint8_t*)msg,
11                           sizeof(msg),
12                           HAL_MAX_DELAY);
13          /* USER CODE BEGIN 3 */
14      }
15      /* USER CODE END 3 */
16  }
17
18 /**
19  * @brief System Clock Configuration
20  * @retval None
21  */

```

The terminal window at the bottom shows the output of the program:

```

ld!lo, World!
Hello World!
Hello, World!llo, World!
Hello, WoHello, World!
Hello, World!
Herld!
Hello, World!
Hello, WorldHello, World!
Helloello, Worldld!
Hello,ello, World!Hello, Worl
Hello, World!
Hell

```

Task 1:

code & output

```
C main.c
Core > Src > C main.c > main

78 /**
79  * @brief The application entry point.
80  * @retval int
81  */
82 void myPrintf(const char *fmt, ...)
83 {
84     char buffer[110];           // Step 1: Output buffer
85     va_list args;              // Step 2: Argument list
86
87     va_start(&args, fmt);       // Initialize argument list
88     vsnprintf(buffer, sizeof(buffer), fmt, args); // Step 3: Format string
89     va_end(&args);              // Clean up argument list
90
91     HAL_UART_Transmit(huart, &uart2,           // Step 4: Send over UART
92                       pData: (uint8_t *)buffer,
93                       Size: strlen(buffer),
94                       Timeout: HAL_MAX_DELAY);
95 }
96 void floatToStr(float value, char *buffer, int precision)
97 {
98     int intPart = (int)value;
99     int fracPart = (int)((value - intPart) * pow(10, precision));
100     if (fracPart < 0) fracPart = -fracPart; // handle negative numbers
101     sprintf(buffer, "%d.%0*d", intPart, precision, fracPart);
102 }
103
104 int main(void)
```

While Loop:

```
C main.c x
Core > Src > C main.c > ...
104 int main(void)
132     MX_USART2_UART_Init();
133     MX_USB_PCD_Init();
134     /* USER CODE BEGIN 2 */
135     int x = 42;
136     float y = 3.14;
137     char y_str[10];
138     /* USER CODE END 2 */
139
140     /* Infinite loop */
141     /* USER CODE BEGIN WHILE */
142     while (1)
143     {
144         floatToStr(value:y, buffer:y_str, precision:2); // convert float to string
145         myPrintf(fmt: "Value of x = %d, y = %s\r\n", x, y_str);
146         HAL_Delay(Delay:1000);
147     }
148     /* USER CODE BEGIN 3 */
149 }
150 /* USER CODE END 3 */
151
```

TASK 2: code & output

```
C main.c • C stm32f3xx_hal.c
Core > Src > C main.c > main
107 int main(void)
138     MX_USART2_UART_Init();
139     MX_USB_PCD_Init();
140     /* USER CODE BEGIN 2 */
141     //TASK #2
142     int a=2;
143     int b=4;
144     int c=(a+b)*(a+b);
145     int d=a*a+b*b+2*a*b;
146
147     //TASK #1
148     // int x = 42;
149     // float y = 3.14;
150     // char y_str[10];
151     /* USER CODE END 2 */
152
153     /* Infinite loop */
154     /* USER CODE BEGIN WHILE */
155     while (1)
156     {
157         myPrintf(fmt: "a = %d, b = %d\r\n", a, b);
158         myPrintf(fmt: "LHS = (a + b)^2 = %d\r\n", c);
159         myPrintf(fmt: "RHS = a^2 + 2ab + b^2 = %d\r\n", d);
160
161         if(c == d)
162             myPrintf(fmt: "Identity holds!\r\n");
163         else
164             myPrintf(fmt: "Identity does NOT hold!\r\n");
165
166         HAL_Delay(Delay:1000); // Wait 1 second before repeating
167     }
168     /* USER CODE BEGIN 3 */
169
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS STM32CUBE RTOS

```
a = 2, b = 4
LHS = (a + b)^2 = 36
RHS = a^2 + 2ab + b^2 = 36
Identity holds!

```

TASK #3

code:

```
C main.c
Core > Src > C main.c > main
107 int main(void)
155 /* USER CODE BEGIN WHILE */
156 while (1)
157 {
158     // Task 3
159     char str[] = "Microcontrollers";    // Original string
160     char encrypted[50];
161     char decrypted[50];
162     int key = 12345;
163     int len = strlen(str);
164
165     // Encryption
166     for(int i = 0; i < len; i++)
167     {
168         encrypted[i] = str[i] + (key % 256);
169     }
170     encrypted[len] = '\0'; // null terminate
171
172     // Decryption
173     for(int i = 0; i < len; i++)
174     {
175         decrypted[i] = encrypted[i] - (key % 256);
176     }
177     decrypted[len] = '\0'; // null terminate
178
179     myPrintf(fmt: "Task 3: String Encryption/Decryption\r\n");
180     myPrintf(fmt: "Original: %s\r\n", str);
181     myPrintf(fmt: "Encrypted: %s\r\n", encrypted);
182     myPrintf(fmt: "Decrypted: %s\r\n", decrypted);
183
184     if(strcmp(str, decrypted) == 0)
185     myPrintf(fmt: "Decryption successful! Strings match.\r\n");
186     else
187     myPrintf(fmt: "Decryption failed! Strings do NOT match.\r\n");
```

output:

```
Original: Microcontrollers
Encrypted: 0000000000000000
Decrypted: Microcontrollers
Decryption successful! Strings match.
Task 3: String Encryption/Decryption
Original: Microcontrollers
Encrypted: 0000000000000000
Decrypted: Microcontrollers
Decryption successful! Strings match.
Task 3: String Encryption/Decryption
Original: Microcontrollers
Encrypted: 0000000000000000
Decrypted: Microcontrollers
Decryption successful! Strings match.
Task 3: String Encryption/Decryption
Original: Microcontrollers
Encrypted: 0000000000000000
Decrypted: Microcontrollers
Decryption successful! Strings match.
```

TASK #4

code:

```
main.c x C stm32f3xx_hal.c
pre > Src > C main.c > main
07 int main(void)
08 // USER CODE BEGIN WHILE
56 while (1)
57 {
58     //TASK #4:
59     int A[2][2] = {[0]={[0]=1, [1]=2}, [1]={[0]=3, [1]=4}}; // Predefined matrix A
60     int B[2][2] = {[0]={[0]=5, [1]=6}, [1]={[0]=7, [1]=8}}; // Predefined matrix B
61     int C[2][2] = {[0]={[0]=0, [1]=0}, [1]={[0]=0, [1]=0}}; // Result matrix C initialized to 0
62
63     // Multiply A and B
64     for(int i = 0; i < 2; i++)
65     {
66         for(int j = 0; j < 2; j++)
67         {
68             C[i][j] = 0; // initialize element
69             for(int k = 0; k < 2; k++)
70             {
71                 C[i][j] += A[i][k] * B[k][j];
72             }
73         }
74     }
75
76     // Print matrices
77     myPrintf(fmt: "Task 4: Matrix Multiplication\r\n");
78
79     // Print Matrix A
80     myPrintf(fmt: "Matrix A:\r\n");
81     for(int i = 0; i < 2; i++)
82     {
83         myPrintf(fmt: "%d\t%d\r\n", A[i][0], A[i][1]);
84     }
85
86     // Print Matrix B
87     myPrintf(fmt: "Matrix B:\r\n");
88     for(int i = 0; i < 2; i++)
89     {
90         myPrintf(fmt: "%d\t%d\r\n", B[i][0], B[i][1]);
91     }
92
93     // Print Result Matrix C
94     myPrintf(fmt: "Matrix C (A*B):\r\n");
95     for(int i = 0; i < 2; i++)
96     {
97         myPrintf(fmt: "%d\t%d\r\n", C[i][0], C[i][1]);
98     }
99     myPrintf(fmt: "\r\n"); // blank line
00 }
```

OUTPUT:

```
Task 4: Matrix Multiplication
Matrix A:
1      2
3      4
Matrix B:
5      6
7      8
Matrix C (A*B):
19     22
43     50
```

TASK #5:

code:

```
int main(void)
{
    while (1)
    {
        //TASK 5
        myPrintf(fmt: "Task 5: Armstrong Numbers between 100 and 999:\r\n");

        for (int num = 100; num <= 999; num++)
        {
            int hundreds = num / 100;           // extract hundreds digit
            int tens      = (num / 10) % 10;     // extract tens digit
            int units     = num % 10;           // extract units digit

            int sum_cubes = hundreds*hundreds*hundreds
                          + tens*tens*tens
                          + units*units*units;

            if(sum_cubes == num)
            {
                myPrintf(fmt: "%d\r\n", num);    // print Armstrong number
            }
        }

        myPrintf(fmt: "\r\n"); // blank line
    }
}
```

Output:

```
100 | | | | int units = num % 10; // extract units digit
167 | | | | |

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS STM32CUBE RTOS

Task 5: Armstrong Numbers between 100 and 999:
153
370
371
407

Task 5: Armstrong Numbers between 100 and 999:
153
370
371
407

Task 5: Armstrong Numbers between 100 and 999:
153
370
371
407

Task 5: Armstrong Numbers between 100 and 999:
153
370
371
407
```

Evaluation Question:

Explain the role of USART2 in UART communication on the STM32F3 Discovery board.

Why must TX and RX lines be crossed when connecting to an external USB-UART module?

USART2 on the STM32F3 board provides the hardware interface for UART communication by handling data transmission and reception, including baud rate, and status control, and is commonly used for communication with a PC or external devices via PA2 (TX) and PA3 (RX).

When connecting to an external USB-UART module, the TX and RX lines must be crossed because the transmitter of one device must connect to the receiver of the other (STM32 TX → USB-UART RX and STM32 RX → USB-UART TX); otherwise, both devices would transmit and listen on the same lines, preventing communication.